

# Comunicação entre Estações de Recarga de Veículo Elétrico e Sistemas de Gerenciamento Intermediadas por Microcontroladores ESP32 e Protocolos de IOT

Ana Carolina Fanhani Stralio<sup>1</sup>, Stephane Beatriz Vale Aguiar<sup>1</sup>, Frank Juergen Knaesel<sup>1</sup>

<sup>1</sup>Instituto Federal de Santa Catarina (IFSC) – Jaraguá do Sul – SC – Brasil

{ana.c2002, stephane.a}@aluno.ifsc.edu.br, frank.knaesel@ifsc.edu.br

**Abstract.** *This paper proposes a communication architecture for electric vehicle charging stations using ESP32 microcontrollers and the CoAP protocol to overcome connectivity limitations of the standard OCPP. The system employs an intermediate Gateway to convert CoAP messages into WebSockets, integrating with a central server. Results demonstrate the viability of the solution for unstable network environments.*

**Resumo.** *Este trabalho propõe uma arquitetura de comunicação para estações de recarga de veículos elétricos utilizando microcontroladores ESP32 e o protocolo CoAP para superar as limitações de conectividade do padrão OCPP. O sistema utiliza um Gateway intermediário para converter mensagens CoAP em WebSockets, integrando-se a um servidor central. Os resultados demonstram a viabilidade da solução para ambientes de rede instáveis.*

## 1. Introdução

A infraestrutura de suporte aos veículos elétricos exige sistemas capazes de operar em condições de rede adversas, onde a persistência do sinal é frequentemente interrompida, impondo desafios críticos à interoperabilidade [Brand et al. 2015] *apud* [de Campos 2024]. O padrão industrial OCPP (Open Charge Point Protocol) utiliza WebSockets sobre TCP para garantir a entrega de mensagens; porém, essa abordagem impõe um custo elevado de conectividade em termos de processamento [de Campos 2024].

Conforme as especificações da RFC 7252 [Shelby et al. 2014], protocolos baseados em UDP, como o CoAP (Constrained Application Protocol), surgem como uma alternativa eficiente para dispositivos restritos. Este trabalho propõe uma arquitetura na qual um microcontrolador ESP32 atua como um Gateway, convertendo pacotes CoAP originados na estação de recarga em mensagens WebSocket compatíveis com sistemas de gerenciamento centralizados. Esta abordagem permite que a estação opere com baixa sobrecarga de rede, enquanto o Gateway assume a responsabilidade pela estabilidade da conexão com o servidor central.

## 2. Fundamentação teórica

### 2.1. Protocolo OCPP

O OCPP (Open Charge Point Protocol) consolida-se como o padrão de interoperabilidade entre o Ponto de Carga (CP) e o Sistema de Gerenciamento Central (CSMS), operando essencialmente através de objetos JSON sobre WebSockets [de Campos 2024]. No entanto,

a camada de transporte TCP, embora robusta, exige um procedimento de estabelecimento de conexão em três etapas e o envio frequente de mensagens de verificação de atividade (heartbeats) para manter o canal de comunicação aberto e funcional [de Campos 2024].

## **2.2. Protocolo CoAP e UDP**

O Constrained Application Protocol (CoAP) é um protocolo especializado em transferência de dados WEB para uso em redes IoT, um de seus objetivos é projetar um protocolo web para os requisitos especiais em ambientes restritos [Barros 2022]. Operando sobre o protocolo orientado a datagrama UDP, o CoAP utiliza um modelo de interação similar ao HTTP (GET, PUT, POST) e permite que o dispositivo envie pacotes sem a necessidade de estabelecer uma conexão persistente, o que o torna ideal para a comunicação entre o carregador e o Gateway local, economizando energia e banda de rede [Barros 2022].

## **2.3. Microcontrolador ESP32**

O uso do ESP32 permite o processamento de borda, onde o Gateway não apenas repassa dados, mas transcodifica protocolos em tempo real [Espressif Systems 2023]. Considerando o suporte nativo a pilhas UDP e TCP/WebSockets, o ESP32 consegue atuar como um servidor CoAP para os carregadores locais e, simultaneamente, como um cliente WebSocket para o servidor na nuvem. Isso permite que dispositivos de baixa potência comuniquem-se com infraestruturas de gestão de recarga.

# **3. Metodologia e implementação técnica**

A implementação do sistema foi dividida em nós de hardware distintos utilizando o framework Arduino em microcontroladores ESP32.

## **3.1. Simulador EVSE**

Utilizando-se de um ESP32, o simulador EVSE gera dados dinâmicos utilizando a biblioteca ArduinoJson. O payload inclui variáveis como tensão e corrente geradas aleatoriamente pela função `gerarMensagemEVSE()` para mimetizar uma recarga ativa. O envio é realizado via comando `coap.put(gatewayIP, 5685, "ocpp/msg", msg.c_str())`, visando o Gateway na rede local. Para garantir a continuidade, o sistema utiliza um objeto Ticker para disparar a função `enviarMensagem` a cada 10 segundos, mantendo o fluxo de dados constante sem bloquear a execução principal.

## **3.2. Gateway**

O Gateway atua como o elemento central da arquitetura, sendo responsável pela ponte de comunicação entre a estação de recarga e o sistema de gerenciamento. Operando na porta 5685, o dispositivo executa a transcodificação de protocolos em tempo real através da função `callbackCoAP`.

A lógica está em extrair o conteúdo dos pacotes CoAP (baseados em UDP) e converte os dados para uma estrutura de string compatível com o protocolo WebSocket (baseado em TCP). Após a conversão, a informação é injetada no cliente WebSocket e enviada ao servidor OCPP por meio do comando `wsOCPP.sendTXT(payload)`.

Para assegurar a integridade da entrega na rede local, o Gateway emite uma resposta de confirmação (ACK) ao simulador EVSE, utilizando o identificador de mensagem (messageid) original do pacote CoAP.

3.3. Sistema de gestão centralizada e monitoramento

O servidor, desenvolvido em ambiente Node.js, atua como o Sistema de Gerenciamento Central (CSMS), estabelecendo um servidor WebSocket que opera na porta 8080. Esta camada é responsável pelo processamento lógico das mensagens normatizadas, como MeterValues (telemetria de consumo) e StatusNotification (estado da estação).

Os dados processados são redirecionados para um Dashboard Web, permitindo o acompanhamento instantâneo da operação. O servidor implementa uma lógica de identificação de origem para permitir o broadcast inteligente: dados originados no microcontrolador ESP32 são convertidos em atualizações visuais para o navegador do usuário, enquanto comandos emitidos pela interface do navegador são capturados pelo servidor e roteados de volta ao Gateway, garantindo o controle remoto da estação de recarga.

4. Resultados e discussões

4.1. Validação de conexão e logs

O sistema demonstrou estabilidade operacional desde a fase de inicialização, operando sem interrupções durante o período de testes. O servidor central, baseado em Node.js, registrou a conexão do Gateway e executou o procedimento de handshake do protocolo WebSocket, emitindo mensagens de boas-vindas que confirmam o sucesso da integração entre o nó intermediário e o CSMS.

No monitor serial do Gateway, o status "Conectado ao servidor OCPP" validou o estabelecimento do túnel de comunicação persistente, essencial para garantir a interoperabilidade técnica entre a estação e o sistema de gerenciamento. Esta etapa é crucial, pois, como apontado por Campos (2024), a interoperabilidade depende da capacidade de manter fluxos de dados contínuos independentemente da tecnologia empregada na rede.

Desta forma, confirma-se que o uso do protocolo CoAP sustenta a integridade dos dados necessária para o monitoramento de recarga. A robustez observada nos registros de log reflete a resiliência do sistema em ambientes de rede adversos, onde a transcodificação eficiente no Gateway garante que as flutuações de sinal não interrompam a lógica de gerenciamento do OCPP.

Tabela 1. Validação de mensagens no sistema

| Mensagem           | Protocolo    | Resposta do servidor                                 | Status  |
|--------------------|--------------|------------------------------------------------------|---------|
| BootNotification   | CoAP para WS | Retorno de aceite ("Accepted")                       | Sucesso |
| MeterValues        | CoAP para WS | Broadcast para dashboard em tempo real               | Sucesso |
| StatusNotification | CoAP para WS | Registro detalhado em arquivo de log (registros.txt) | Sucesso |
| OCPPResponse       | WS para CoAP | Confirmação enviada do servidor para o gateway       | Sucesso |

#### 4.2. Análise de latência e resiliência

A latência foi calculada entre o envio no Gateway e o registro no Servidor. Os dados da tabela 2, obtidos a partir de testes, demonstram uma operação de alta performance para redes locais.

**Tabela 2. Latência capturada entre mensagens consecutivas**

| Mensagem capturada | Protocolo de origem | Resposta do servidor                                  | Status final |
|--------------------|---------------------|-------------------------------------------------------|--------------|
| BootNotification   | CoAP para WS        | Retorno de aceite ("Accepted")                        | Sucesso      |
| MeterValues        | CoAP para WS        | <i>Broadcast para dashboard em tempo real</i>         | Sucesso      |
| StatusNotification | CoAP para WS        | Registro detalhado em arquivo de log (registros.txt)  | Sucesso      |
| OCPPResponse       | WS para CoAP        | Confirmação enviada do servidor para o <i>gateway</i> | Sucesso      |

Como evidenciado na Tabela 2, os intervalos entre as mensagens seguem a configuração de 10 segundos estabelecida no código do simulador (envioPeriodico.attach(10, enviarMensagem)). O desvio padrão da latência foi mínimo, demonstrando que a ponte CoAP-WebSocket implementada no ESP32 não adiciona gargalos significativos ao sistema. É importante salientar que a latência estimada refere-se ao tempo de transcodificação no Gateway somado ao tráfego de rede local.

Este comportamento é crucial para aplicações em tempo real, como o monitoramento de recarga, onde atrasos podem causar erros na tarifação ou falhas na proteção de sobrecorrente da estação. Nos testes de log do servidor, é possível observar que as mensagens são recebidas e logadas instantaneamente, contando com valores como: [16:03:23] MeterValues e [16:03:33] MeterValues, percebidos em testes.

#### 5. Conclusão

Este trabalho validou uma arquitetura eficiente para aumentar a resiliência de infraestruturas de recarga elétrica. O uso do protocolo CoAP provou reduzir o overhead computacional no carregador, enquanto o Gateway ESP32 garantiu a transparência necessária para a integração com servidores OCPP padrão via WebSockets. Por sua vez, a implementação de um dashboard em tempo real permitiu a visualização clara da eficácia da transcodificação de protocolos.

#### Referências

- Barros, A. S. (2022). Análise de desempenho dos protocolos coap, mqtt-sn e http em smart homes. Analisa a eficiência do CoAP em cenários de redes instáveis.
- Brand, A., Iacob, M.-E., and van Sinderen, M. J. (2015). *Interoperability Architecture for Electric Mobility*. S.l.: s.n.

de Campos, G. G. (2024). Estudo de implementac~ao dos protocolos ocpp e ocppi para interoperabilidade entre redes de carregamento de ve´iculos el´etricos.

Espressif Systems (2023). *ESP32 Technical Reference Manual*.

Shelby, Z., Hartke, K., and Bormann, C. (2014). Rfc 7252: The constrained application protocol (coap). Technical report, IETF.